高并发与大数量面试题

1. 流量过大时如何避免系统崩溃，用户能得到正确的信息？

核心思路

从限流、熔断、降级、隔离四个层面回答，配合具体项目案例。

参考答案

我主要从四个层面来设计：

1. 限流（控制流量进入）

- 在网关层做请求限流，常用算法：计数器、滑动窗口、漏桶、令牌桶
- 我在 [项目名称] 中使用 Redis 滑动窗口限流，按手机号+IP+设备指纹多维度分桶，限制单位时间内请求频率
- 滑动窗口比固定窗口更平滑，避免边界突刺问题

2. 熔断（保护下游服务）

- 当下游服务响应慢或错误率升高时，主动熔断快速失败
- 我使用 .NET 的 Polly 库做熔断配置，设置失败阈值、熔断时长、半开恢复
- 熔断后返回兜底数据，不至于完全不可用

3. 降级（保证核心功能）

- 非核心功能暂时关闭，保证核心流程可用
- 例如：秒杀时关闭实时排行榜、关闭详情页评论功能
- 降级后返回缓存数据或友好提示

4. 隔离（避免雪崩）

- 线程隔离：不同业务用不同线程池，一个业务出问题不影响其他
- 资源隔离：数据库连接池分层、读写分离
- 服务隔离：核心服务独立部署，非核心服务独立部署

5. 用户体验保证

- 返回友好提示：“系统繁忙，请稍后重试”
- 请求进入排队系统，返回排队号码和预计等待时间
- 使用异步处理，用户请求先受理，后台处理完成后通知

效果指标

- 可用性保持在 99% 以上
- 熔断降级时有明确的错误码和提示信息
- 核心流程（登录、下单）不受影响

---

2. 数据量过大（上亿条）如何快速查询？

核心思路

从存储方案、查询优化、架构设计三个层面回答。

参考答案

1. 数据库层面优化

- 分库分表：按时间或业务维度水平拆分，我做过按月份分表的方案
- 读写分离：主库写入，从库读取，分担查询压力
- 索引优化：建立合适的复合索引，覆盖索引减少回表
- 分区表（Partition）：MySQL 分区、SQL Server 分区，按分区查询减少扫描范围

2. 查询优化

- 避免 SELECT *，只查询必要字段
- 分页查询改用游标分页（基于 ID 或时间戳），避免深度分页
- 预计算：常用统计指标提前算好，用空间换时间
- 查询改写：避免全表扫描，尽量走索引

3. 引入中间件

- ElasticSearch：适合复杂条件检索和全文搜索，亿级数据查询毫秒级响应
- 我在 [项目] 中用 ES 做商品检索，查询时间从 2s 降到 50ms
- ClickHouse：适合 OLAP 分析场景，亿级数据聚合查询秒级出结果
- Redis：热点数据放缓存，减少数据库压力

4. 离线处理

- 数据量大但实时性要求不高时，用定时任务提前处理
- 预计算报表结果、预聚合数据
- 物化视图自动刷新

具体案例

我优化过报表查询从 1000ms 到 200ms：

- 梳理慢 SQL，建立复合索引
- 拆分复杂 JOIN，预计算中间结果
- 引入 Redis 缓存热点报表数据

---

3. 如何设计一个秒杀系统？

核心思路

秒杀的核心问题是：瞬间流量大、库存有限、不能超卖。围绕”拦截流量”、”保护下单”、”保证库存”来回答。

参考答案

1. 流量拦截（多级限流）

- CDN 层：静态资源+用户防刷
- 网关层：IP 限流、请求削峰
- 业务层：验证码校验，拦截刷请求

2. 库存校验（关键环节）

- 预扣库存： Redis 扣减库存，成功后再写数据库
- 乐观锁： 数据库更新时用 version 或条件 where stock > 0
- 防止超卖： 数据库唯一索引约束（订单号）

3. 请求削峰

- 消息队列： 把下单请求写入 MQ，异步处理
- 排队机制： 返回排队号码，分批通知用户结果
- 限流策略： 令牌桶算法控制进入下单页的请求数

4. 注意事项

- 独立部署： 秒杀服务与其他业务隔离，避免影响正常业务
- 热点数据： 秒杀商品信息提前缓存到 Redis
- 熔断降级： 超出系统承载时返回”秒杀已结束”

---

4. 如何保证缓存与数据库的一致性？

核心思路

缓存和数据库一致性的核心问题是：先删缓存还是先更新数据库？回答时要说明问题根源和解决方案。

参考答案

问题本质

- 先删缓存再更新数据库： 可能导致其他请求读到旧缓存
- 先更新数据库再删缓存： 可能导致其他请求读到旧数据（概率较低）

推荐方案：延迟双删

1. 先删除缓存
2. 再更新数据库
3. 延迟几百毫秒后再次删除缓存

异步方案：订阅 binlog

- 使用 Canal 监听数据库变化，异步更新缓存
- 保证最终一致性，适合对实时性要求不高的场景

一致性要求高的场景

- 用分布式锁： Redisson 锁，保证更新缓存和数据库的原子性
- 直接放弃缓存： 强一致性要求时直接查数据库

我的实践

- 一般场景：用延迟双删，代码简单，效果够用
- 高一致性场景：用订阅 binlog 或分布式锁

---

5. 如何保证接口的幂等性？

核心思路

幂等性指多次执行结果一致。重点回答：如何识别重复请求、如何保证只执行一次。

参考答案

什么是幂等

- GET：天然幂等
- PUT：多次更新为相同值，幂等
- DELETE：多次删除同一资源，幂等
- POST：非幂等，需要处理

常用实现方案

1. 唯一请求标识（推荐）

- 前端生成 UUID 或雪花算法 ID 作为 token
- 用 Redis SETNX 检查是否处理过，处理后删除或标记
- 适用于：下单、支付等关键操作

2. 数据库唯一约束

- 利用唯一索引防止重复插入
- 适用于：创建订单、用户注册等

3. 乐观锁

- 表中加 version 字段，更新时检查 version
- 适用于：更新操作

4. 状态机校验

- 订单状态：待支付→已支付→已完成
- 只有在正确状态下才能流转，防止重复处理

我的实践

- 短信验证码：用 Redis 限重（同手机号 N 分钟内只能发一次）
- 下单：用 Redis SETNX 检查订单 token
- 支付回调：用数据库唯一约束+状态机

---

6. 如何设计一个分布式 ID 生成器？

核心思路

分布式 ID 的要求：唯一、趋势递增、高可用、性能好。常见方案有雪花算法、UUID、数据库自增等。

参考答案

常见方案对比

方案	唯一性	趋势递增	性能	依赖
UUID	✅	❌	高	无
数据库自增	✅	✅	一般	数据库
雪花算法	✅	✅	高	无
Redis INCR	✅	✅	高	Redis

雪花算法（推荐）

```
1位符号位 + 41位时间戳 + 10位机器ID + 12位序列号
```

- 41 位时间戳：可支撑 69 年
- 10 位机器 ID：可支持 1024 个节点
- 12 位序列号：每毫秒每个节点可生成 4096 个 ID

我的实践

- 用雪花算法自研 ID 生成服务，无外部依赖
- 机器 ID 用配置文件或注册中心分配
- 部署多实例保证高可用

注意事项

- 时钟回拨：雪花算法依赖时间，需处理时钟回拨问题
- 机器 ID 分配：需要提前规划，避免冲突

---

7. 如何处理分布式事务？

核心思路

分布式事务的核心问题是：如何保证多个服务要么都成功，要么都失败。回答时说明适用场景。

参考答案

常见方案

1. 2PC（两阶段提交）

- 准备阶段：各服务执行但不提交
- 提交阶段：都成功后统一提交
- 问题：同步阻塞、单点故障、数据不一致风险

2. TCC（Try-Confirm-Cancel）

- Try：预留资源
- Confirm：确认执行
- Cancel：取消预留
- 优点：性能较好
- 缺点：侵入业务代码，需要实现三个接口

3. 可靠消息最终一致性

- 发送方：本地事务 + 消息表，事务成功后发送消息
- 消费方：消费消息后执行本地事务，重试直到成功
- 适用于：一致性要求不那么高的场景

4. Saga 模式

- 将事务拆分为多个子事务，按顺序执行
- 每个子事务有对应的补偿操作
- 适用于：长流程业务

我的实践

- 强一致性：用 TCC（如：库存扣减+订单创建）
- 最终一致性：用可靠消息（如：下单后发送通知）
- 根据业务场景选择，不追求过度设计

---

8. 如何设计一个抢票/抢号系统?

核心思路

和秒杀类似但有区别：抢票是”先到先得”，库存可能有很多个，且需要考虑占位（座位、号码）。

参考答案

核心问题

- 多用户同时抢同一资源
- 需要按时间顺序保证公平
- 防止重复抢

方案设计

1. 抢锁机制

- Redis 分布式锁：抢购时先抢锁，保证串行
- 抢成功后占位（Redis SET 记录用户已抢的号码/座位）
- 设置过期时间，超时释放

2. 库存扣减

- Redis DECR 原子扣减库存，扣减为负则抢失败
- 库存扣减成功后，异步写入数据库

3. 排队和通知

- 抢票请求进入消息队列
- 消费队列处理抢票逻辑
- 抢成功后通过 WebSocket 或短信通知用户

4. 防止重复

- 用户 ID + 资源 ID 联合唯一索引
- Redis SET 记录用户已抢资源

注意事项

- 需考虑限流防刷
- 数据库和 Redis 数据一致性
- 抢成功后超时未支付需释放资源

面试加分项

1. **能说出具体数字**：如”查询时间从 X 降到 Y”
2. **结合项目案例**：不要只说理论，要说有实际落地经验
3. **提到监控和告警**：说明有完整的观测体系
4. **考虑成本**：提到技术选型时的成本考量